Terminal ×

```
(base) codio@banditruby-questbeyond:~/workspace$
```

Guide ×

Collapse

Deploying to Heroku

Creating the Heroku App

You should already have an account with Heroku (the CLI application is already installed). Login to your Heroku account with the following command.

```
heroku login -i
```

Remember, you cannot use your Heroku password with the CLI application. Instead, use the API key, which is found under you account settings on the Heroku site.

Next, create your Heroku application for this project. The name should be unique so adding something like your inititals to the template below may help. In addition, we want to add the Python buildpack to our Heroku application. Make a note of the URL Heroku generates for your application. We will need this to check if our Flask app is running properly.

```
heroku create ascii-flask-<your initials> --buildpack heroku/python
```

▼ **Unique names**

> *If the name you chose for your app is not unique, you will see a message that this name is already taken. You will have to run* `heroku create` *again with a different name. You may have to do this a few times until you find a unique name.*

The last thing we need to do is set the Heroku stack to run a container. Use the following command to do this; be sure to substitute the name of your Heroku application in the template below.

```
heroku stack:set container -a <heroku app name>
```

Heroku YAML File and Flask App

```
touch heroku.yml
```

Heroku follows the commands set forth in the `heroku.yml` file. Add the snippet of code to the top-left panel. This tells Heroku to build the Docker container according to the Dockerfile. From there, Heroku will use the entry point in the Dockerfile to start the container. Use the link to open the file and copy/paste the code into it.

Open heroku.yml

```
build:
  docker:
    web: Dockerfile
```

The last thing we need to do is make some changes to our Flask app for deployment on the internet. Use the link below to open this file.

Open main.py

Import the `os` module so we can set an environment variable for the script.

```
import os
from flask import Flask, render_template, request
from ascii_art import create_ascii
```

At the bottom of the Python file, get the `PORT` environment variable and use it when starting the Flask app. Then change the debug mode to `False`.

```
if __name__ == '__main__':
    port = os.environ.get("PORT", 5000)
    app.run(host='0.0.0.0', port=port, debug=False)
```

The last steps are to use git to push our code to a Heroku git repository. From there, Heroku will deploy our Flask app to the internet. Use the link to open the terminal. Then add our changes.

Open terminal

```
git add .
```

Next, we need to commit our changes. Use the `-m` flag to write a message about the commit. Since this is our first commit, “initial commit” is a sufficient message. If you make any future changes, use descriptive commit messages to help you remember the changes you made.

```
git commit -m "initial commit"
```

Finally, push our changes to the git repository. This will kick of a series of events that build our Docker image, create a container, and ultimately lead to our Flask app running on the internet.

```
git push heroku master
```

Once this process finishes, your webpage is live. Use the Heroku URL to visit the site. If you cannot remember your URL, use the following command in the terminal.

```
heroku open
```

If you were to run this command on your local computer (and not in Codio), Heroku would open the browser and load your Flask app. However, Heroku throws an error because it cannot open a browser inside Codio. However, it does list the URL. Click on the link, and your webpage will load in a new tab.

```
▸    Error opening web browser.
▸    Error: Exited with code 3
▸
▸    Manually visit https://ascii-flask.herokuapp.com/ in your browser.
```

Lab Question

Fill in the blanks below.

- Heroku is a platform as a service.
- Docker is a platform for building containers.

The correct answers are:

- **Heroku** is a platform as a service.
- **Docker** is a platform for building containers.

While these two platforms are often used in conjunction with one another, they are quite different. Docker is a platform for building containers. Heroku is a platform as a service, which means we can easily host the container for others.

Check It!

Next